



**University of Arkansas – CSCE Department
Capstone II – Final Project – Spring 2026**

Wearable Balance Sensor

**Kai Pokrandt, Dhairya Dodia, Tyler Bean, Walter Sarmiento, Siddarth Suresh
Babu, Dylan Flanagan**

Abstract

We propose a wearable insole-and-sock system with a large pressure-sensing pad and an embedded microcontroller (MCU) that acquires plantar-pressure data to assess gait, balance, and fall risk. The system produces a concise Fall Risk Assessment PDF while supporting near-real-time collection and analysis of pressure and motion data to assist gait and balance evaluation. The solution targets physical therapists and clinical staff who need objective, repeatable measures of balance and gait to screen for risks, guide therapy, and document outcomes.

1.0 Problem

Falls are a leading cause of injury and hospitalization among older adults and patients with neurological or musculoskeletal conditions. Clinical screening tools (e.g., TUG, Berg Balance Scale) rely on manual timing or qualitative scoring, which can be inconsistent across raters and visits. Objective gait labs are accurate but expensive and impractical for routine clinic visits. Clinicians need an affordable, portable, and objective tool that captures spatiotemporal gait metrics and center-of-pressure dynamics during typical activities (sit-to-stand, quiet stance, straight-line walk, turning), and produces standardized reports for the chart.

Epic EMR is used to manage patient data and clinical workflows for hospitals and healthcare providers. It provides hospitals and clinics with tools for storing patient data, documenting visits, ordering tests, reviewing diagnostics, and supporting clinical workflows. The platform includes APIs such as FHIR, a data transfer standard that was created by the Health Level Seven (HL7) international organization and SMART-on-FHIR that allow external medical devices and software systems to integrate securely with electronic charts. Epic EMR will not be necessary for the full implementation, instead, it will focus on near-real-time pressure data and accelerometer readings to assess risk to generate the PDF.

Without such a tool, fall risks may go unrecognized, progress is hard to quantify, and care teams lack timely data at the point of care.

2.0 Objective

The primary objective of the project is to design, build, and validate a wearable insole system capable of accurately measuring and analyzing plantar pressure data to assess gait and balance.

The system will employ a single large pressure sensor pad distributed across the sole of the foot to capture real-time load distribution and aggregate readings at a rate of 100ms. Using this data, the system will compute key gait and balance metrics such as step length, cadence, stance-to-swing ratio, asymmetry indices, and center-of-pressure dynamics, including path length, sway velocity, and ellipse area. The output will be a concise one-page Fall Risk Assessment Report in PDF format, which can be imported into the EMR system as part of clinical documentation. Report import into Epic EMR is no longer necessary. The system will operate wirelessly, powered by a rechargeable battery that supports at least eight hours of continuous use, ensuring practicality in clinical environments. Finally, the design will adhere to HIPAA-compliant data security standards through local encryption, de-identification of test data, and secure device storage protocols.

3.0 Background

Falls represent one of the most significant causes of injury, hospitalization, and loss of independence among older adults and individuals with neurological or musculoskeletal impairments. Current clinical assessment methods, such as the Timed Up and Go (TUG) [7] and Berg Balance Scale [8], rely on subjective scoring and manual timing, which can vary significantly between evaluators and across sessions. While gait laboratories provide gold-standard objective data, they are expensive, space-intensive, and impractical for routine use in most healthcare settings. Therefore, a need exists for a portable, affordable, and objective device that can capture spatiotemporal gait metrics and CoP dynamics during common clinical tasks such as quiet stance, sit-to-stand, and straight-line walking. By producing standardized digital reports compatible with Epic EMR systems, the proposed device addresses a critical gap in clinical workflow efficiency and fall-risk management.

3.1 Key Concepts

The design is grounded in several key technical and biomedical concepts.

Plantar-pressure sensing involves the use of force-sensitive resistors or piezoresistive fabrics positioned beneath the heel, metatarsal heads, hallux, and midfoot regions to quantify load distribution and calculate CoP. The *center of pressure* is computed as a weighted centroid of sensor forces, providing valuable insight into postural sway and gait stability through derived features such as total path length, mean velocity, and sway ellipse area. On the hardware side, the system relies on an embedded low-power microcontroller (such as an ARM Cortex-M) equipped with analog-to-digital converters and multiplexers to process sensor data, while communication is handled via *Bluetooth Low Energy (BLE)*. A rechargeable lithium-ion battery supplies power, with a built-in fuel gauge and charging circuitry to ensure reliability. Signal processing techniques such as baseline drift correction, per-sensor calibration, thresholding, smoothing, and outlier rejection will improve data fidelity.

3.2 Related Work

Several research and commercial efforts have explored the use of wearable insoles and balance platforms for gait and postural assessment. Early foundational work by Rosenbaum and Becker (1997) established the clinical utility of plantar-pressure distribution measurements for diagnosing gait abnormalities and balance deficits [1].

Subsequent studies demonstrated that multi-sensor insoles could accurately quantify center-of-pressure trajectories and load distributions under dynamic conditions.

More recent developments in wearable gait analysis have leveraged inertial and pressure sensors for automated stride detection and spatiotemporal parameter estimation. Rampp et al. (2015) validated an inertial sensor-based approach for calculating stride parameters in geriatric populations, confirming the feasibility of unobtrusive, clinic-ready systems for objective gait monitoring [5]. Complementing these findings, Nascimento et al. (2020) reviewed a wide array of wearable sensor systems for physical rehabilitation and health monitoring, emphasizing the importance of integrated data processing, wireless communication, and user comfort in the design of medical-grade wearable platforms [6].

Commercial systems such as Moticon's OpenGo Science have demonstrated practical wireless in-shoe pressure measurement with embedded data logging and Bluetooth connectivity [9]. However, most existing systems remain constrained by proprietary data formats, a lack of open algorithmic transparency, and limited interoperability within clinical record systems such as Epic EMR. The proposed wearable balance sensor expands upon these works by prioritizing FHIR-based interoperability, transparent computation of gait and balance features, and generation of standardized clinical documentation directly suitable for EMR integration.

4.0 Design

The wearable balance sensor system will be composed of three major subsystems: the insole hardware, designed by the Electrical Engineering team, the host software application, InsolePro, designed by the CSCE team, and the sock housing developed by the BMEG team. The insole will contain a single large pressure sensor pad that detects variations in plantar load during gait and stance. These signals will be amplified and digitized by an onboard microcontroller, which will transmit the data wirelessly via BLE to a host application running on a PC. The host software will process raw sensor data to extract clinically meaningful metrics and generate standardized reports.

4.1 Use Cases

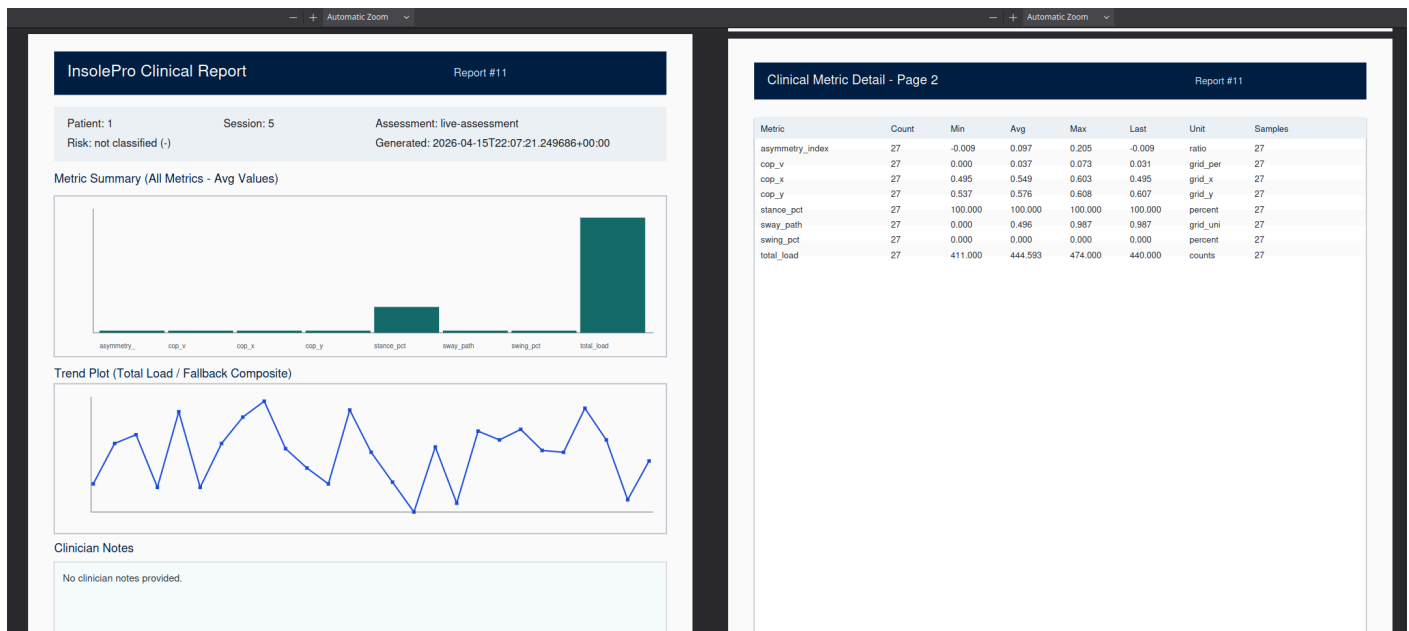
- Physical Therapist (PT): Runs 30-120 second tests (quiet stance, TUG, 10-meter walk), reviews live trends, generates PDF, uploads to Epic.
- Physician/Provider: Reviews PDF in Epic; later, reviews live stream/graphs inside Epic widget.
- Clinical Assistant: Pairs device, selects patient, starts/stops sessions.
- Research/QA (optional): Exports de-identified CSV for analysis.
- UC-1: Pair device and run Quiet Stance to compute CoP sway metrics, then save and export PDF.
- UC-2: Run 10-meter Walk to compute cadence, stance/swing %, asymmetry, and export to PDF.

4.2 Requirements

The system requirements encompass functional, interface, performance, and safety specifications designed to ensure accuracy, usability, and compliance with healthcare standards.

4.2.1 Functional Requirements

The insole will sample a single large pressure sensor pad at a rate of 100ms, producing frames containing both measurement data and status information. Embedded algorithms will detect gait events such as heel-strike and toe-off to compute metrics including cadence, step-time variability, stance-to-swing ratio, double-support duration, asymmetry index, CoP path length, velocity, and sway ellipse area. From the computed data, the host application will use algorithms provided by the Biomedical Engineering team to compute a Fall Risk Assessment report summarizing patient information, key metrics, and an overall stratification of fall risk (low, moderate, high). The application will use a database to store data during the session to process the raw data using algorithms. The Fall Risk Assessment PDF



is shown below:

4.2.2 Interface Requirements

The communication architecture will rely on Bluetooth Low Energy (BLE) 4.x for data transmission to a host device such as a clinical workstation. USB-C connectivity will enable device charging and firmware updates. The host software will maintain encrypted local storage and provide options for exporting PDF reports. The user will be able to monitor and view various patient's saved metrics for assessment through the UI. The UI will be designed with HTML and CSS with Django.

4.2.3 Performance Requirements

The system will target, but not require, an end-to-end latency, defined as the time between the final sensor sample and the report generation, of under thirty seconds. Synchronization between sensors and the host application will maintain drift below ten milliseconds per minute. Power optimization will allow at least five hours of continuous clinical use, and the system will recharge to 80% capacity within two hours. These parameters ensure the device meets practical clinical throughput and usability standards.

4.2.4 Regulatory/Safety Requirements

All materials in contact with the user will be biocompatible and skin-safe, conforming to relevant IEC 60601-1 electrical safety standards and offering basic ingress protection against sweat and moisture. Data handling and storage will strictly adhere to HIPAA regulations, ensuring encrypted local storage, secure Bluetooth pairing, and de-identified data export. No cloud data transmission will occur without prior encryption and consent protocols.

4.3 High-Level Architecture

The overall architecture integrates hardware and software into a streamlined data flow. The insole hardware consists of a large force-sensitive pad connected to an analog front end that feeds into an MCU equipped with an analog-to-digital conversion and multiplexing capabilities. The MCU communicates with the host application through BLE while being powered by a rechargeable battery. The host software, developed in Python, HTML, and CSS, handles BLE data reception, calibration, feature extraction, and report generation.

The Electrical Engineering team is solely responsible for all hardware development, including sensor selection, PCB design, microcontroller integration, power regulation, and embedded electronics.

The Biomedical Engineering team is responsible for defining system-level clinical requirements, validating the design, and specifying all algorithms required for gait analysis, center-of-pressure computation, and fall-risk classification.

The Computer Science and Computer Engineering team is responsible entirely for firmware implementation, host software, user interface, Bluetooth communication, database, PDF generation, and FHIR integration.

4.4 Detailed Architecture

The embedded subsystem, implemented in C, includes drivers for analog-to-digital conversion, multiplexing, BLE communication, and battery monitoring. Data sampling is governed by a real-time scheduler, which performs packetization using efficient custom ASCII-formatted sensor data. On the host side, the Python application consists of a Bluetooth data acquisition layer, parsing and bridge scripts, and a Django-based backend for session management and data storage. BLE communication is handled using Python scripts built with Bleak, which connect to the embedded device, subscribe to sensor notifications, and parse ASCII-formatted sensor frames transmitted over BLE. The host software currently includes components for BLE communication, packet parsing, session management, frame

upload, and database-backed storage of patient sessions and raw sensor data. A Python bridge script converts incoming pressure and accelerometer data into structured frame records and forwards them through REST endpoints to the backend, where sessions and measurements are stored in an SQL-based database. A clinician-facing interface allows patient information to be managed, sessions to be recorded, and collected data to be reviewed through the web application.

4.4.1 Embedded Firmware Layer

Sensor data is acquired on the embedded system using an STM32 microcontroller that scans a custom pressure sensing matrix and reads inertial data from the LIS3DH accelerometer. The pressure interface is implemented as a discrete sensing matrix rather than a continuous piezoresistive pad. In the current prototype, 12 sensing locations are active and transmitted.

Pressure and inertial data are packaged in an ASCII-formatted telemetry frame and transmitted periodically (approximately every 100 ms) over Bluetooth Low Energy through the RN4871 module operating in Transparent UART mode.

Each frame follows this format:

AX:<int>,AY:<int>,AZ:<int>,

S0:<v0>,<v1>,

S1:<v0>,<v1>,<v2>,<v3>,

S2:<v0>,<v1>,<v2>,<v3>,

S3:<v0>,<v1>\r\n

An example frame is as follows:

AX:-576,AY:-384,AZ:15680,

S0:539,405,

S1:850,1186,933,1292,

S2:388,482,368,526,

S3:262,326

AX, AY, AZ are the raw signed accelerometer readings corresponding to X, Y, Z axes from the LIS3DH sensor. The toe of the foot has two sensors inside the pad, the ball has 4 sensors, the midfoot has 4, and the heel has 2 sensors, corresponding to S0, S1, S2, S3 in the ASCII frames.

The embedded firmware currently implements GPIO row/column scanning for pressure acquisition, ADC sampling of pressure sensor nodes, Microcontroller communication with the LIS3DH accelerometer, UART communication with the RN4871 BLE module, and ASCII packet generation with periodic BLE transmission.

4.4.2 Host Application Data Pipeline

Each BLE frame is transmitted as an ASCII sensor frame using the format defined in Section 4.4.1. The inputs to the host application are Bluetooth packets containing

accelerometer readings and pressure matrix data emitted from the embedded layer. These frames are received through the BLE interface, parsed on the host, and converted into structured sensor records for storage and downstream analysis.

Incoming ASCII packets are decoded into a logical pressure matrix representation, combined with accelerometer data, timestamped on the host, and stored as session frames in the SQL-backed backend.

1. Decode the ASCII-format into a 2D pressure map.
2. Calibrate the device's profile, with help from the Biomedical team.
3. Segment the foot regions (heel/midfoot/forefoot/hallux).
4. Compute CoP for every frame.
5. Detect gait events (heel-strike, toe-off) via region activation patterns.
6. Calculate metrics including cadence, stance/swing ratio, load asymmetry, CoP path length, and sway velocity.
7. Classify risk level, as defined by the Biomedical team.

4.4.3 EMR Integration Layer

We were fully prepared for integrating Epic EMR into this project, however, after communicating with the Biomedical Engineering team, we have assessed that the value of this feature is not required for the function they desire. Phase 1 is the automated PDF upload using the Epic Document Import. Phase 2 is the SMART-on-FHIR OAuth2 with Observations and DiagnosticReports. A FHIR Observation represents a single clinical measurement, such as CoP path length or sway velocity. A FHIR DiagnosticReport aggregates multiple Observations and may embed the PDF report. Example FHIR Observation JSON:

```
{
  "resourceType": "Observation",
  "status": "final",
  "code": { "text": "CoP Path Length" },
  "subject": { "reference": "Patient/{id}" },
  "effectiveDateTime": "2025-01-01T12:00:00Z",
  "valueQuantity": { "value": 10, "unit": "cm" }
}
```

4.5 Data Management and Storage

The system employs a relational SQL database through the Django backend to store patient information, session metadata, and raw sensor frame data collected from the device. The database supports session organization, raw data persistence, and later offline analysis of recorded gait and pressure data. Simplification of the original database was needed given the hardware restrictions of the custom PCB, STEPPA.

Each session's raw sensor data may also be exported to CSV or similar analysis formats to support transparency, debugging, and future algorithm development.

The core of the current schema consists of patient, session, and raw frame data.

The **patients** table stores a unique **patient_id** primary key for each subject along with associated demographic information managed through the web application. Patient records are linked to recorded sessions through relational foreign key references.

Each use of the device generates a new record in the **sessions** table. This table uses **session_id** as its primary key and includes a foreign key reference to **patient_id**, associating each recording session with a specific patient. Session records include metadata such as start and end timestamps and the device used during acquisition.

Sensor data collected during a session is stored in the **raw_frames** table. Each sample frame is identified by a **frame_id** primary key and linked back to its parent session through **session_id**.

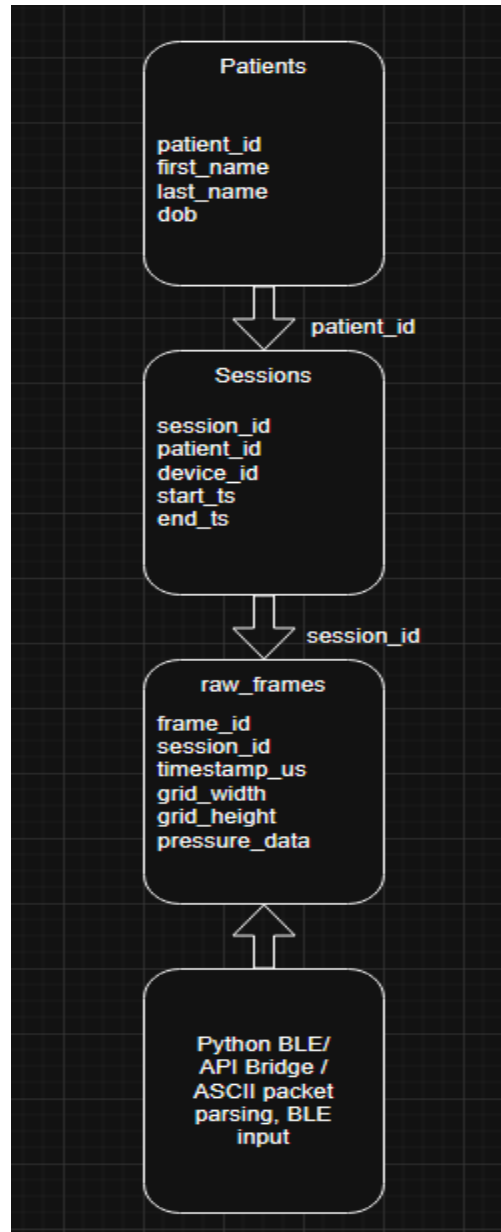
Each frame stores a host-side timestamp, pressure grid dimensions, encoded pressure sensor values, and supporting frame metadata such as battery or flags.

Pressure data transmitted from the embedded system is received as ASCII-formatted sensor frames over BLE, parsed by the host application, and stored in structured form within the backend. This supports preservation of raw sensor measurements.

4.6 Python Application Development Plan

Because the insole design has transitioned from multiple discrete pressure sensors to a single large-area pressure-sensing pad at the behest of the Electrical Engineering team, the Python host application will play a more substantial computational role in segmenting, interpolating, and decoding the pressure distribution into clinically meaningful metrics. The host app, implemented in Python, will function as the primary computation engine responsible for real-time BLE data acquisition, calibration, feature extraction, visualization, and report generation. The host application will include real-time visualization modules for pressure heatmaps and CoP trajectories, providing clinicians with immediate feedback during testing. The plan will be done in 11 steps:

- Define Clinical Metrics (Biomedical Team)
- Finalize Sensor Grid specs (Electrical Team)
- Firmware Development (Electrical + CSCE Team)
- Host Software Architecture (CSCE)
- Algorithm Implementation (CSCE, based on Biomedical Team models)
- Database Integration (CSCE)
- Report Generation (CSCE)
- Validation & Bench testing (ALL)
- Pilot Testing



4.7 Sensor-Pad-Specific Data Pipeline

The single-pad design, which uses a continuous piezoresistive fabric or force-sensitive sheet, outputs a 2D pressure map sampled through row-column scanning rather than eight individual sensor voltages. The BLE packets from the MCU will contain a flattened grid, encoded in a structured ASCII format. The Python app reconstructs the grid, performs denoising, and computes the following derived maps. This pipeline replaces earlier per-cell thresholding with continuous-map analysis, increasing system robustness across foot sizes and patient variability.

- 1) Spatial Pressure Map - a 2D matrix reconstructed from each sample frame (100ms).
- 2) Center-of-Pressure - computed using weighted centroid formulas over the grid.

- 3) Gait Event Detection - heel-strike and toe-off detected by spatial patterns rather than discrete sensors.
- 4) Segmentation Masks - automatic algorithms partition the grid into anatomical regions (heel / midfoot / forefoot / hallux).

4.8 Mathematical Foundations

The system derives clinically relevant gait and balance metrics from the two-dimensional plantar pressure map produced by the sensor pad. Each sample frame represents a pressure distribution across the sensing grid, which is reconstructed in the host application as a matrix $P(x,y)$ where each element corresponds to the measured pressure at coordinate (x,y) on the insole. Using this pressure distribution, several features are computed.

The center of Pressure represents the effective point of application of the resultant ground reaction force across the foot. It is calculated using the weighted centroid of the pressure distribution across the grid.

Let F_i represent the pressure value measured at sensor location (x_i, y_i) . The CoP coordinates are computed as:

$$CoP_x = \frac{\sum_{i=1}^N x_i F_i}{\sum_{i=1}^N F_i} \quad CoP_y = \frac{\sum_{i=1}^N y_i F_i}{\sum_{i=1}^N F_i}$$

Where N represents the total number of sensing elements in the pressure grid. This calculation produces an instantaneous center of pressure for each frame of data. Tracking the CoP position over time forms a trajectory that reflects postural sway and weight transfer during gait.

The total CoP path length is a measure of postural stability during a test session. It represents the cumulative distance traveled by the CoP trajectory over time.

If $CoP(t)$ represents the center of pressure at time t , the path length L over T frames is:

$$L = \sum_{t=2}^T \sqrt{(CoP_x(t) - CoP_x(t-1))^2 + (CoP_y(t) - CoP_y(t-1))^2}$$

Larger path lengths typically correspond to instability during a quiet stance.

Sway velocity measures the average speed of CoP movement during a balance test. It is computed as:

$$V_{sway} = \frac{L}{T_{session}}$$

where L is the total CoP path length, and $T_{session}$ is the duration of the measurement interval. Higher sway velocity values can indicate impaired balance control.

Cadence represents the number of steps taken per minute during walking. Gait events such as heel-strike are detected using spatial pressure activation patterns across the heel region of the sensor grid. If N_{steps} represent the number of detected steps during a measurement period

of duration T , cadence is computed as:

$$Cadence = \frac{60 \times N_{steps}}{T}$$

To calculate the Stance-to-Swing ratio, the team must find the length of the stance and swing phases. The stance phase corresponds to the portion of the gait cycle where the foot remains in contact with the ground, while the swing phase represents the portion where the foot is airborne. If T_{stance} represents the average stance duration and T_{swing} represents the swing duration, the stance-to-swing ratio is:

$$SSR = \frac{T_{stance}}{T_{swing}}$$

If using two balance sensors, we must quantify the differences between the left and right limbs through an asymmetry index. If M_L and M_R represent a measured metric (such as stance time or peak load) for the left and right sides, respectively, the asymmetry index is defined as:

$$AI = \frac{|M_L - M_R|}{0.5(M_L + M_R)} \times 100$$

Higher asymmetry values may indicate gait issues or compensatory movement patterns.

These mathematical principles form a results panel that is shown below:



4.9 Software Modules

The host application is implemented as a Python and Django-based software stack running through Docker and organized around BLE acquisition, sensor-frame parsing, session management, backend storage, and web-based review tools. A large rework of the previous structure was needed to function and integrate with the hardware provided by the ELEG team.

The current implementation consists of several core software components.

The BLE communication layer manages Bluetooth Low Energy communication between the sensing hardware and the host application using Python scripts built with Bleak. This layer handles device discovery, connection management, BLE notification subscriptions, and reception of ASCII-formatted sensor frames transmitted by the embedded device. Incoming notifications are buffered, assembled into complete newline-delimited sensor frames, and forwarded to parsing and storage components.

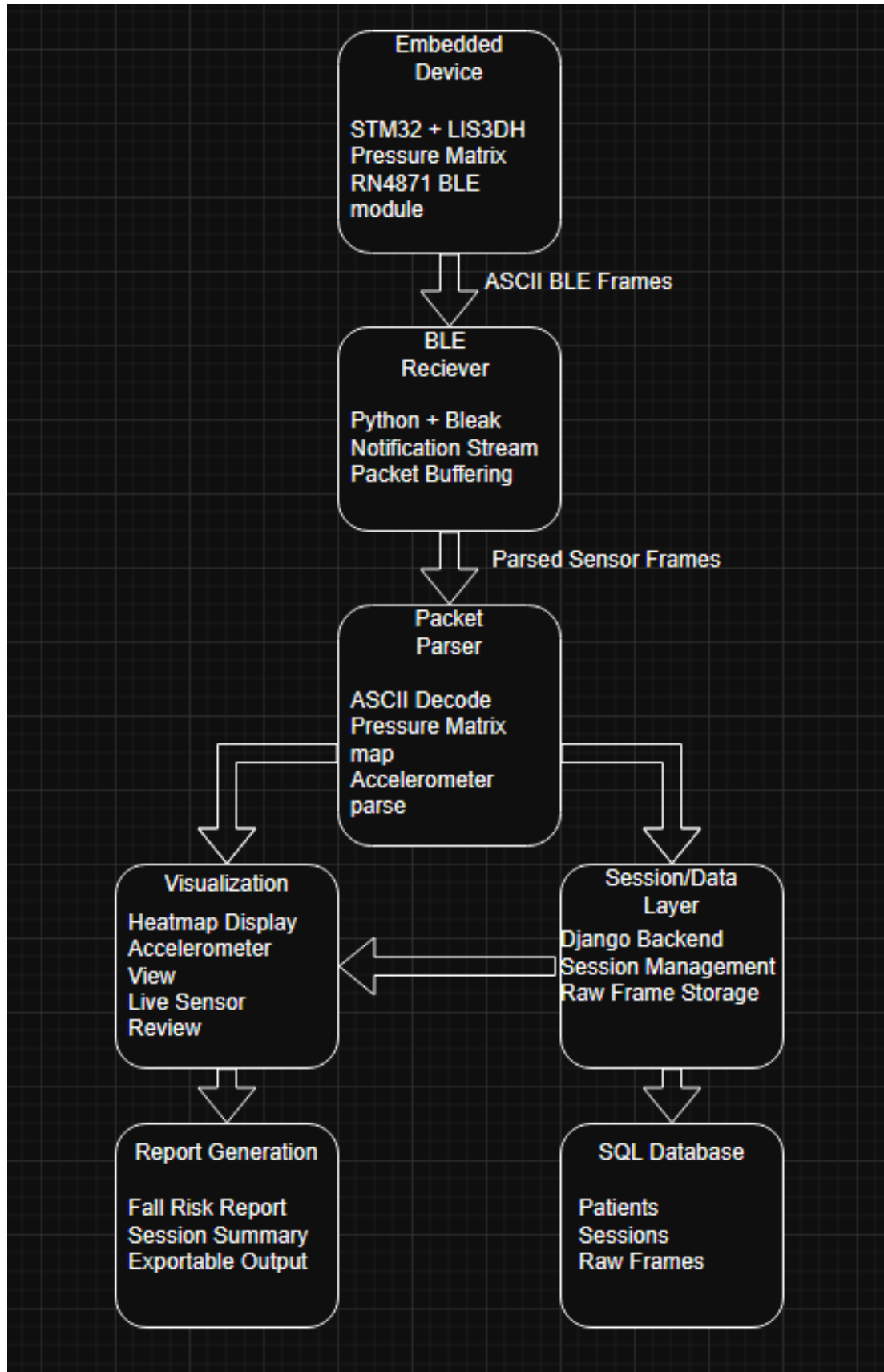
A parsing component decodes the ASCII sensor frame format defined in Section 4.4.1 into structured accelerometer and pressure matrix data. This layer reconstructs the logical pressure matrix, associates host-side timestamps with each frame, formats sensor data for backend storage, and supports raw data export for offline analysis.

Session management is handled through the Django backend and associated REST APIs. This layer supports patient records, session creation and closure, raw frame ingestion, SQL-backed storage of session data, and retrieval and review of stored sessions. A Python BLE-to-API bridge coordinates live sensor acquisition with backend session and frame upload workflows.

The host application includes a web-based interface implemented through Django for session review and data management. The project supports real-time heatmapping of the foot via the pressure pad, and fall status through accelerometer values. These will then export into a single Fall Risk Assessment PDF for clinical review.

Database operations are managed through Django models and backend services rather than a standalone custom database model that was planned previously. The database stores patient information, session metadata, and raw sensor frame data.

The visualization of the program architecture is shown below:



4.10 Hardware Modules

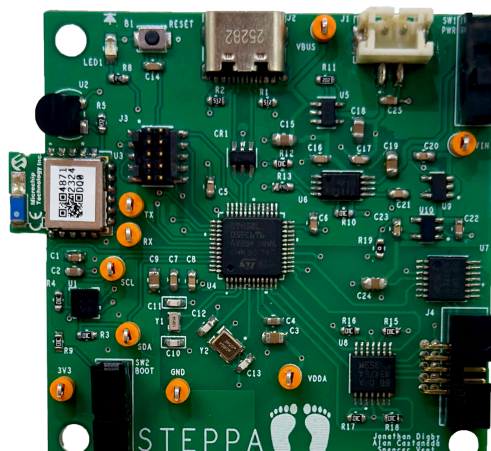
The wearable balance sensor system relies on a custom-designed printed circuit board (PCB) that integrates sensing, computation, and wireless communication capabilities into a compact embedded platform suitable for placement within the insole assembly. The PCB is responsible for acquiring motion and pressure-related measurements, performing low-level signal processing, and transmitting collected data to the host application through Bluetooth Low Energy (BLE).

The primary sensing component on the board is the [LIS3DHTR three-axis accelerometer](#) manufactured by STMicroelectronics. This MEMS-based accelerometer provides high-resolution measurements of acceleration along the X, Y, and Z axes and supports selectable measurement ranges and sampling rates. The accelerometer is used to capture dynamic motion characteristics of the foot during gait, including step impact, foot orientation changes, and movement transitions. These measurements complement the plantar pressure sensing pad by providing inertial context that helps identify gait phases and detect sudden instability events that may correlate with increased fall risk.

At the core of the embedded system is the [STM32L052C6T6 microcontroller](#), an ultra-low-power ARM Cortex-M0+ based MCU. This microcontroller performs several essential tasks within the hardware subsystem. It manages data acquisition from the pressure-sensing pad and accelerometer through its analog-to-digital converters and digital communication interfaces. The MCU executes embedded firmware routines responsible for sensor polling, timestamp generation, packet formation, and battery monitoring. Due to its low power consumption and integrated peripherals, the STM32L052C6T6 enables continuous sampling while maintaining efficient energy usage suitable for a wearable medical device operating on battery power.

Wireless communication is provided by the [RN4871 Bluetooth Low Energy module](#), produced by Microchip Technology. This module enables the PCB to transmit collected sensor data to the host application running on a nearby workstation or tablet. The RN4871 supports BLE 4.x communication protocols and provides a compact, low-power wireless interface suitable for embedded medical devices. The module operates as a BLE peripheral device, advertising services that allow the host software to connect and receive real-time sensor data streams. By using BLE, the system avoids wired connections and allows clinicians to operate the device without restricting patient movement.

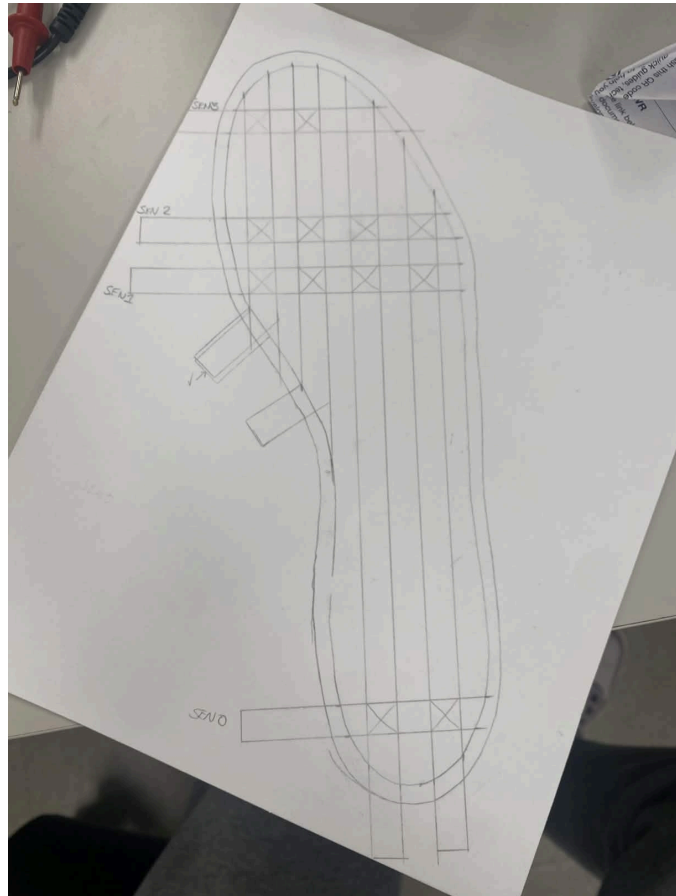
Additional supporting hardware components on the PCB include voltage regulation circuitry, battery management components, and communication interfaces that allow firmware programming and debugging. Together, these modules form a compact embedded platform capable of continuously collecting sensor data, performing preliminary processing, and reliably transmitting measurement packets to the host application. The PCB is shown below.



A central component of the hardware subsystem is the custom pressure-sensing pad, which is designed specifically for this project to capture plantar pressure distribution across the foot. The sensor pad is constructed using a resistive sensing architecture consisting of two conductive layers separated by a pressure-sensitive resistive material. When force is applied to the pad, the resistance between the conductive layers changes proportionally to the applied pressure. By measuring these resistance changes, the system can determine the magnitude and distribution of force across the sensing surface.

To enable spatial pressure detection, the conductive layers are arranged in a row-column grid configuration. Conductive strips on one layer are sequentially energized with a voltage signal, while the corresponding strips on the opposing layer are read by the microcontroller's analog-to-digital converters (ADC). This scanning technique allows the system to detect pressure at multiple intersection points within the grid while minimizing the number of required electrical connections. As a result, the design supports a high-resolution pressure map without requiring a large number of individual sensor wires, improving both reliability and manufacturability of the insole hardware.

The sensor pad is connected to the system's embedded controller, which manages the scanning sequence and digitizes the measured signals. By repeatedly scanning each row and column at a rate between 50 and 100Hz, the device produces a continuous two-dimensional pressure map representing the user's plantar load distribution over time. These measurements form the primary dataset used for computing center-of-pressure trajectories, gait phase detection, and balance stability metrics. The sensor pad and corresponding grid is shown below.



4.11 Software Infrastructure

In addition to the primary hardware platform, the system incorporates a supporting software infrastructure designed to ensure reliable development, testing, and data processing even when the physical hardware is unavailable. This infrastructure currently consists of a pseudo-metric generator, a backend processing framework, and several supporting utility modules.

The pseudo-metric generator is a simulation tool developed to emulate sensor output generated by the wearable hardware. The generator produces synthetic sensor data that follows the same Bluetooth packet structure as the packets transmitted by the embedded microcontroller. This allows the team to test the host software pipeline before the hardware system is fully operational.

The generator produces simulated pressure grids at a configurable sampling rate of 100ms. Each frame represents a simulated pressure distribution across the insole sensor grid. Several movement scenarios are supported, including quiet standing, walking, and instability disturbances. These scenarios approximate realistic plantar loading patterns and center-of-pressure trajectories during gait cycles. Each generated pressure frame is converted into the ASCII packet structure defined in Section 4.4.1. The generator can either stream packets directly to the host application or write them to files for later processing and testing. This tool allows the team to verify the correctness of the decoding pipeline, feature extraction algorithms, and database integration without requiring a physical device to be connected.

The backend processing system acts as the central computation engine for ingesting and analyzing incoming sensor data. It can operate in two modes: live processing of Bluetooth packets received from the hardware device or offline replay of previously recorded sessions generated by the pseudo-metric generator. When the backend receives a stream of packets, the following modular processing pipeline is executed:

- 1) Packet decoding: Incoming packets are parsed according to the frame format described in Section 4.4.1.
- 2) Pressure grid reconstruction: The flattened pressure data array is converted back into a two-dimensional pressure grid corresponding to the layout of the sensor pad.
- 3) Filtering and calibration: Noise reduction and calibration adjustments are applied to normalize the pressure values.
- 4) Feature extraction: Biomechanical metrics such as center-of-pressure position, sway path length, sway velocity, cadence, stance-to-swing ratio, and asymmetry indices are computed from the reconstructed pressure maps.
- 5) Risk classification: The derived features are aggregated to estimate fall-risk indicators for the session.

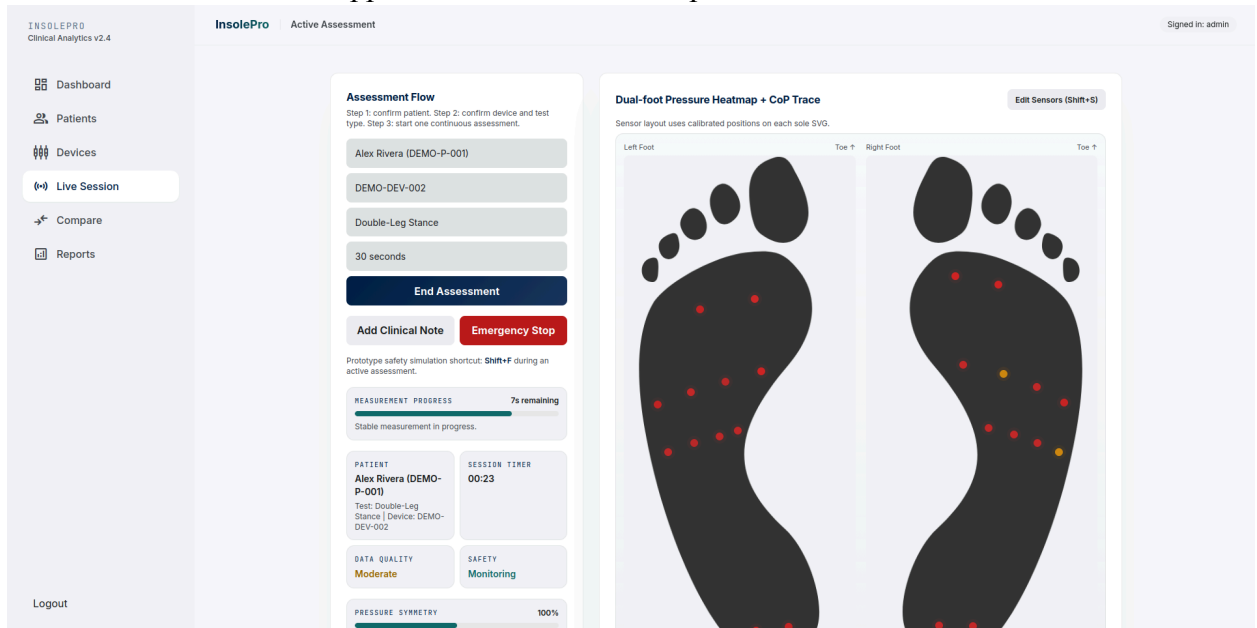
The backend processes the generated session and produces summary metrics such as session duration, center of pressure velocity, cadence, asymmetry index, and a fall-risk score. This development infrastructure allows the pipeline to be validated independently of hardware progress and ensures that the host application can be tested with realistic sensor data prior to full device integration. Generated sessions can be viewed and tested through the Django frontend.

5.0 Development Plan

5.1 Tasks

- Requirements finalization
During the first phase, the team will refine and document all system requirements in consultation with stakeholders and biomedical collaborators. This includes specifying hardware performance targets, BLE communication parameters, and sampling rates.
- Sensor pad selection & procurement; insole mechanical design.
The team will research and procure a suitable pressure-sensing pad, such as piezoresistive fabric. Each sensor pad's range, linearity, and durability will be evaluated. The Biomedical Engineering group will design the insole layout and sock interface using CAD tools, ensuring accurate sensor pad placement under critical plantar regions. A flexible PCB or wiring harness will be drafted for integration with the microcontroller unit (MCU) by the Electrical Engineering team.
- MCU bring-up; ADC/mux driver; BLE link.
Embedded work will focus on bringing up the chosen MCU (e.g. an ARM Cortex-M-based board). Tasks include configuring ADC channels and multiplexers, implementing BLE communication, and establishing sampling routines in an RTOS or superloop environment. Firmware will handle packet framing, calibration lookup tables, and basic battery monitoring. Successful bring-up will be verified through bench tests with simulated sensor inputs via programmable voltage sources. This is the responsibility of the Electrical Engineering team.
- Power subsystem (charger, gauge) and battery tests.
A parallel hardware effort will design and test the power subsystem, including a Li-ion battery, charging circuit, and fuel-gauge IC. The group will characterize the runtime under typical sampling loads and verify the recharge time and thermal behavior. Efficiency and safety testing will ensure compliance with IEC 60601-1 principles and sweat-resistant insulation. This is the responsibility of the Electrical Engineering team.
- Calibration jig + scripts; per-cell curve fitting.
A calibration jig will be fabricated to characterize each sensor's force-voltage response. The team will then implement baseline correction, smoothing, and threshold detection algorithms to derive gait events such as heel-strike and toe-off. Accuracy will be assessed against reference loads and validated on a small group of volunteer test subjects (non-PHI data).
- Host BLE receiver (Python) + data logger.
The desktop host application, written in Python, will be extended with BLE 4.x connectivity to receive streamed data packets. The streamed data packets referenced here follow the ASCII packet format defined in **Section 4.4.1**. Logging, timestamp synchronization, and basic visualization modules will be added. Data will be stored in a local database and can be exported to CSV for analysis.

A screenshot of the Host App with the related heatmap shown below:



- Feature extraction algorithms (Heel-Strike/Toe-Off, cadence, CoP).
Software modules will compute gait and balance features, including cadence, step-time variability, stance-to-swing ratio, CoP path length, and sway ellipse area. Algorithms will be validated against known gait sequences to ensure temporal accuracy within ± 5 ms. Derived metrics will later feed into the PDF report and EMR upload pipeline. Algorithms will be derived and produced by the Biomedical Engineering team.
- PDF report generator
A reporting engine will compile patient metadata and computed metrics into a concise one-page Fall Risk Assessment. Layout and formatting will follow clinic standards for readability and HIPAA compliance. The report generator will automatically embed calibration status and battery level summaries for traceability.
- Bench tests, unit tests, and latency/power profiling.
Comprehensive unit and integration tests will be run on both firmware and host software. Bench tests will measure latency, synchronization drift, and power consumption. Upon validation, a short pilot will be conducted using de-identified datasets. User feedback will guide final refinements.
- Documentation, installation, and a user guide will be provided, showcasing all features and how to use them.

5.2 Development Progress

As of April 2026, the project has reached its final stages. We have successfully navigated the research, design, and implementation phases, culminating in the integration of hardware and software components. The project was organized into parallel tracks for hardware, firmware,

and software development, which ensured consistent consensus and allowed for iterative improvements throughout the development lifecycle.

5.2.1 Hardware Platform Development:

The hardware development phase began with comprehensive sensor research (January 20–26, 2026), where we selected a custom-made piezoresistive pressure-sensing pad, paired with the accelerometer, microcontroller, and Bluetooth Low Energy (BLE) module. Hardware specifications and board designs were finalized concurrently (January 20–26, 2026).

Following component selection, the ELEG team conducted extensive firmware research (January 20–February 4, 2026). The final hardware integration phase (April 10–21, 2026) addressed significant challenges; after initial difficulties in board functionality, the ELEG team successfully performed final repairs. As of April 21, 2026, the board is fully integrated, successfully communicating with the host system to produce heatmaps, with ongoing efforts focused on accelerometer calibration.

5.2.2 Backend Infrastructure Development

In parallel with hardware development, the backend infrastructure was established to ensure robust data management:

- **Signal Processing:** Initial solutions for mapping voltage readings to pressure values were completed by January 28, 2026.
- **Database Architecture:** The custom relational schema (including tables for patients, sessions, and calibration profiles) was finalized and implemented between February 10 and February 19, 2026.
- **Docker/Django Integration:** To modernize the deployment and frontend interaction, the system was redesigned using Docker and the Django framework (March 12–24, 2026).

5.2.3 Host Application Development

The host application serves as the central hub for data ingestion and visualization. Development proceeded through several modular stages:

- **Core Communication & Simulation:** Environment setup and sensor simulation tools were completed by February 18, 2026. The sensor simulator was critical for software development prior to final hardware availability.
- **Decoding & Analysis:** The `pad_decoder.py` was completed by February 25, 2026, to process raw data. The `gait_features.py` module, which extracts metrics from the processed signals, was finalized on February 19, 2026.
- **Feature Extraction & Reporting:** Advanced feature extraction was completed by March 31, 2026, alongside the `report_pdf.py` module, which currently generates clinical documents using synthetic and integrated test data.

- UI/UX: The frontend design and visualization interface were finalized on March 30, 2026.

Note: EMR integration tasks were ultimately deemed unnecessary for the scope of the BMEG requirements and were not pursued.

5.2.4 Upcoming Development Tasks

The final phase of the project involves comprehensive system evaluation and documentation:

- System Evaluation: Performance profiling and pilot testing are currently in progress (started April 10, 2026). A review session with the BMEG team confirmed that the software successfully meets clinical expectations, despite the final-hour hardware adjustments.
- Documentation: Final documentation and the project poster were developed throughout April to encapsulate the project's technical findings and outcomes.

5.2.5 Development Status Summary

The project has successfully transitioned from a simulated environment to a functional, hardware-integrated system. Core infrastructure, including the database, host processing pipeline, and reporting modules, is fully operational. While hardware challenges required last-minute adjustments, the system is now generating valid pressure data, and final testing and documentation efforts are concluding on schedule as of April 22, 2026.

5.3 Schedule

Task Description	Assigned Members	Est Start	Est End	Actual Start	Actual End
Sensor Research	Kai, Dhairya	1/20	1/26	1/20	1/26
Signal Processing	Kai, Dhairya	1/20	1/28	1/20	1/28
Hardware Specifications	ELEG/ Syd, Walter	1/20	1/26	1/20	1/26
Firmware Research	ELEG/Kai, Dhairya	1/20	2/4	1/20	2/4
Database Setup	Kai, Syd, Dylan	2/10	2/19	2/10	2/19
Host App Development	Tyler, Dylan	2/10	2/18	2/10	2/18

Wearable Balance Sensor

Sensor Simulator	Kai, Dhairya	2/10	2/16	2/10	2/16
Gait Features	Syd, Tyler	2/10	2/19	2/10	2/19
Docker/Django Setup	Syd, Walter	3/12	3/24	3/12	3/24
Decoder	Kai, Syd	2/10	2/25	2/10	2/25
UI/UX Frontend	Syd, Walter	3/10	3/30	3/10	3/30
Feature Extraction	Tyler, Dylan	3/10	3/31	3/10	3/31
Report Generator	Tyler, Dylan	3/10	3/31	3/10	3/31
EMR Integration (p1)	Syd, Kai	3/25	4/3	3/25	4/3
EMR Integration (p2)	Syd, Kai, Tyler	3/25	4/6	3/25	4/3
Evaluation	CSCE/ELEG/BM EG	4/10	4/16	4/10	4/21
Documentation	Kai, Dhairya, Dylan	3/30	4/16	4/2	4/22
Hardware Integration	Kai, Dhairya, Syd	4/10	4/16	4/10	4/23
Poster	Walter, Syd	4/2	4/13	4/2	4/13
Final Report	CSCE	1/20	4/22	TBD	4/22

5.4 Deliverables – The final submission will include all hardware, firmware, software, documentation, and verification materials required to demonstrate a fully functional wearable balance-sensing system. Each deliverable is intended to provide traceability from requirements through validation and to support future continuation of the project into clinical or commercial phases.

- **Design Document:** A comprehensive engineering design report will describe every major hardware and software subsystem of the wearable balance sensor. This document will include detailed block diagrams of the insole electronics, MCU circuit, BLE interface, and power subsystem; schematics and PCB layouts; firmware module descriptions; host-application architecture. This deliverable serves as the authoritative technical reference for future deployment and maintenance.

- **Software:** An application that can process the data received from the hardware and process it using algorithms to detect the fall risk of the patient. The software will have the ability to produce a PDF report of the session in which the wearable sensor was used. The report will include all the metrics recorded and the fall risk assessment.
- **Database scheme and initial data:** A lightweight relational database schema is included to manage patient-session metadata and session results. Entity-relationship diagrams and example SQL dumps will accompany the schema.
- **Hardware Prototype:** A fully assembled insole-and-sock prototype will be delivered, incorporating the selected pressure sensor pad, MCU, BLE transceiver, and battery subsystem. The prototype will be mechanically robust enough for repeated clinical demonstrations and will include labeled test points and connectors for debugging. This prototype will serve as the basis for all calibration, testing, and demonstration activities described in the validation section.
- **Embedded Firmware:** The complete MCU firmware, written in embedded C, will be included with inline documentation and version control history. The firmware implements sensor pad sampling, ADC multiplexing, BLE packet transmission, calibration-table loading, battery monitoring, and firmware update functionality. Source code will be provided to build instructions and unit-test results verifying timing accuracy and data-integrity compliance.
- **User Manual and Installation Guide:** A detailed guide will describe how to assemble, calibrate, and operate the wearable balance sensor system. Instructions will cover firmware flashing, host-software setup, Bluetooth pairing, running test sessions, generating reports, and troubleshooting common errors. Safety considerations and maintenance procedures will be clearly stated.

6.0 Key Personnel

Kai Pokrandt - Pokrandt is a senior Computer Engineering major in the Electrical Engineering and Computer Science Department at the University of Arkansas. He has worked on several recreational projects, including a Private Equity Dashboard, a Stock Portfolio optimization tool, Packet Steganography and Firewalls, and AI/ML Prompt sandboxes to test token usage. Responsible for research, design, architecture, scheduling, and writing this whole document.

Dhairya Dodia - Dodia is a senior majoring in Computer Engineering at the University of Arkansas. He has worked on multiple hardware and software projects. Some of the projects include building a working processor unit, a price management system, and programming a microcontroller robot from scratch. Responsible for app development, documentation, and integrating APIs.

Dylan Flanagan - Flanagan is a senior Computer Engineering major at the University of Arkansas. He has completed various internships that relate to his major, most notably working as a front-end and back-end developer for a major health insurance company. Some of his personal projects he has worked on are a music creation AI, an FPGA-based digital system, a line

following robot, and various webpages/online games. Responsible for research, design, and evaluation.

Tyler Bean - Bean is a senior Computer Science major in the Computer Science department at the University of Arkansas. He has completed an internship at National Beef Packing Company, contributing to database management, API integration, and UI development. Responsible for research, MCU bring-up, Epic integration, and documentation.

Walter Sarmiento - Sarmiento is a senior Computer Science major in the Department of Electrical Engineering and Computer Science at the University of Arkansas. He is the founder and CEO of Swaply, a student-only marketplace redefining how college students buy, sell, and rent within campus communities. With a strong background in product management and design, he has experience in user-centered design, product cycles, UX/UI design in Figma, and development in SwiftUI. He is responsible for firmware development, features, testing, profiling, and documentation.

Siddarth Suresh Babu - Siddarth Suresh is a senior Computer Science major at the University of Arkansas and the CTO/co-founder of Swaply. He has strong experience in AWS, backend architecture, and full-stack development, specializing in scalable system design and cloud infrastructure. For this capstone, he is responsible for firmware development, feature extraction, and testing & profiling, ensuring seamless performance and reliability across all system components.

Sponsors:

Morten Jensen - Biomedical Engineering instructor, responsible for helping organize the biomedical engineering capstone team.

Robert Saunders - Electrical Engineering instructor, responsible for helping organize the electrical engineering capstone team.

Working with the Biomedical Engineering and Electrical Engineering teams, for a total of 14 members.

Biomedical Engineering Team: Eric Hamaker, Drew Hamilton, Lauren McKinney, Bree Scott, Vaughn Spurlin, Sam Stephens, Hannah Vaughan

Electrical Engineering Team: Alan Castaneda, Jonathan Digby, Spencer Vent

7.0 Facilities and Equipment

The development and testing of the Wearable Balance Sensor system will leverage university-provided and team-acquired resources across the University of Arkansas's Electrical Engineering and Computer Science (EECS) and Biomedical Engineering (BME) departments. Hardware development and testing will be conducted primarily by the Electrical Engineering team. Overall design of the system will be handled by the Biomedical Engineering team. Software development, data processing, and EMR integration testing will be carried out through our team, using high performance workstations equipped with Python, MATLAB, C/C++ toolchains (ARM GCC), and Git-based version control. Database and EMR integration modules will be developed and tested in a sandbox environment hosted on secure systems. This environment allows simulation of Epic FHIR/HL7 endpoints for testing document upload and

SMART-on-FHIR data exchange without accessing live patient data. Additional resources include SQL, BLE communication libraries (BlueZ/PyBluez), and PDF generation frameworks (ReportLab and FPDF). All data processing, storage, and EMR integration will be executed on clinic-owned local workstations or tablets. No cloud storage or external hosting will be used. All components run on-premise to maintain HIPAA compliance. To ensure compliance with safety and performance standards, the team will perform electrical and battery testing using battery analyzers, current probes, and thermal imaging tools. All documentation, schematics, and source code will be managed using GitHub, Box cloud repositories (only documentation and code), Microsoft Teams, and GroupMe, shared between the EECS and BME departments.

8.0 References

- [1] Rosenbaum, D., Becker, H. P. (1997). “Plantar pressure distribution measurements. Technical background and clinical applications.” *Gait & Posture*, 5(1), 35-56.
<https://onlinelibrary.wiley.com/doi/abs/10.1046/j.1460-9584.1997.00043.x>
- [2] U.S. Department of Health and Human Services. *Health Insurance Portability and Accountability Act (HIPAA) of 1996*. Accessed October 2025.
<https://www.hhs.gov/hipaa/index.html>
- [3] International Electrotechnical Commission (IEC). *IEC 60601-1: Medical Electrical Equipment - Part 1: General Requirements for Basic Safety and Essential Performance*. IEC Standard 60601-1, 3rd Edition, 2012. <https://webstore.iec.ch/en/publication/2603>
- [4] Epic Systems Corporation. *Epic FHIR API and SMART on FHIR Developer Documentation*. Accessed October 2025. <https://fhir.epic.com/Documentation?docId=developerguidelines>
- [5] Rampp, A., Barth, J., Schulein, S., et al. (2014). “Inertial sensor-based stride parameter calculation from gait sequences in geriatric patients.” *Sensors*, 15(7), 14509-14523.
<https://pubmed.ncbi.nlm.nih.gov/25389237/>
- [6] Nascimento, L. M. S. do, Bonfati, L. V., La Banca Freitas, M., Mendes Jr., J. J. A., & Siqueira, H. V. (2020). *Sensors and Systems for Physical Rehabilitation and Health Monitoring - A Review*. *Sensors* (Basel, Switzerland), 20(15), 4063.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC7436073/>
- [7] Cleveland Clinic. (2021). Berg Balance Test (Berg Balance Scale). Accessed October 2025.
<https://my.clevelandclinic.org/health/diagnostics/22090-berg-balance-scale>
- [8] Thompson Health. (n.d.). Functional Tests: Timed Up and Go (TUG) Test. Accessed October 2025.
https://www.thompsonhealth.com/Portals/0/_RehabilitationServices/PT%20Mgmt%20of%20Knee/Functional_Tests.pdf
- [9] Moticon GmbH. (2024). *OpenGo Science: Wireless In-Shoe Sensor Technology for Gait Analysis*. Munich, Germany.
https://www.thompsonhealth.com/Portals/0/_RehabilitationServices/PT%20Mgmt%20of%20Knee/Functional_Tests.pdfany. <https://moticon.com/opengo/sensor-insoles>